

Slowly Changing Dim Notebook

New notebook

SCD Type 2 Production Template

Below is a **clean, production-ready, Databricks/Fabric** notebook template that performs **Slowly Changing Dimension Type 2 (SCD2)** handling using **Delta Lake MERGE**. This is the **industry-standard** implementation used in enterprise lakehouse architectures.

It supports:

- **Insert new dimension rows**
- **Expire old rows** when a change is detected
- **Insert new SCD2 version** with updated fields
- **Natural key-based matching**
- **Versioning and effective dates**

You can use this notebook as a **production template** immediately.

Michael Obideyi @

SCD Type 2 — Delta Lake Notebook Template

1. Parameters

If you want to get current Lakehouse info (friendly name)

```
In [ ]: import os
        from notebookutils import mssparkutils
        from pyspark.sql.functions import col, to_date, when, lit, regexp_replace, to_timestamp
        from pyspark.sql.types import TimestampType
        import uuid

        print(os.getcwd())

        row = spark.sql("SELECT current_catalog(), current_database()").collect()[0]
        print(row)

        catalog = row[0]
        lakehouse = row[1]

        current_lakehouse_id = row[0]
        print("ID : ", current_lakehouse_id)

        print("Catalog:", catalog)
        print("Lakehouse Name:", lakehouse)

        #print("Lakehouse ID:", lakehouse_id)
        #print("Lakehouse Name:", Lh["name"])
```

```
# from config setting
#print(spark.conf.get("spark.sql.catalogImplementation"))
#print(spark.conf.get("spark.sql.defaultDatabase")) #not ca
```

```
In [ ]: row = spark.sql("SELECT current_database()").collect()[0]
print("Spark Lakehouse ID:", row[0])

from notebookutils import mssparkutils

lakehouse_id = spark.sql(
    "SELECT current_database()"
).collect()[0][0]

lh = mssparkutils.lakehouse.get(lakehouse_id)

#print("Lakehouse name:", lh["name"])
print("Lakehouse id:", lh["id"])
print("Workspace id:", lh["workspaceId"])
```

Create the table that holds the process run log where tracking and scheduling of job runs - if does not exist

```
In [3]: %%sql
CREATE TABLE IF NOT EXISTS etl_run_log (
    batch_id          STRING,
    notebook_name     STRING,
    target_table      STRING,
    start_time        TIMESTAMP,
    end_time          TIMESTAMP,
    status            STRING,
    rows_source       BIGINT,
    rows_inserted     BIGINT,
    rows_updated      BIGINT,
    error_message     STRING
)
USING DELTA;
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 5, Finished, Available, Finished, False)

Out[3]: <Spark SQL result set with 0 rows and 0 fields>

Source table (staging area / raw feed) and Target Location

```
In [4]: Run_Debug = False # Flag to track debug and reset processes

source_path = "Files/staging/customers"

# Target SCD2 dimension table
target_path = "Files/dim/customer_dim"
target_table = "gold_dim_customer"
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 6, Finished, Available, Finished, False)

We want to retrieve the module name dynamically to track in our audit/log of the SCD run process

- Captured the executing notebook identifier dynamically at runtime using the Spark **environment context**.
- Recorded this metadata within the audit log to ensure execution traceability and governance compliance

```
In [ ]: from notebookutils import mssparkutils

ctx = mssparkutils.runtime.context

# print(ctx) - we can get some attributes of the fabric environment
print(ctx.get("currentNotebookName"))
```

```
In [6]: batch_id = str(uuid.uuid4())
notebook_name = ctx.get("currentNotebookName")    # "e.g scd_type2_lite"
target_table = "gold_dim_customer"
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 8, Finished, Available, Finished, False)

```
In [ ]: spark.sql(f"""
INSERT INTO etl_run_log
SELECT
    '{batch_id}',
    '{notebook_name}',
    '{target_table}',
    current_timestamp(),
    NULL,
    'RUNNING',
    NULL, NULL, NULL
""")
```

```
In [ ]: spark.sql("select * from etl_run_log").show()
```

Business Key (could be a compound key - multiple columns for row uniqueness)

```
In [9]: # Natural key for SCD2
business_key = "customer_id"
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 11, Finished, Available, Finished, False)

Type 2 SCD Change Columns to track - So any change in any of these columns would flag a new 'insert' for the customer

and "expire" their previous row (populate effective_end date

```
In [10]: # Columns used to detect changes (non-key columns)
scd_columns = ["customer_name", "customer_status", "address"]
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 12, Finished, Available, Finished, False)

```
In [11]: # Metadata columns for SCD2
effective_start = "effective_start_date"
effective_end = "effective_end_date"
is_current = "is_current"
created_at = "created_at"
updated_at = "updated_at"
batch_id_flag = "batch_id"
```

```
now = "current_timestamp()"
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 13, Finished, Available, Finished, False)

2. Load Incoming Staging Data - load to memory in temporary table

- Add surrogate key to source for insertion - manage real robust Type 2 mechanism - track history

```
In [12]: from pyspark.sql.functions import monotonically_increasing_id, current_timestamp

df_bronze = spark.read.format("delta").load(source_path).orderBy("customer_id")
df_bronze.write.format("delta").mode("overwrite").saveAsTable("bronze_customers")

df_silver = (
    spark.table("bronze_customers")\
    .withColumn("is_current", lit(True))\
    .withColumn("created_at", current_timestamp())\
    .withColumn("updated_at", lit(None).cast(TimestampType()))\
    # Add surrogate key to source for insertion
    .withColumn("row_sk", monotonically_increasing_id())
    #.withColumn(effective_start, lit(None).cast(TimestampType()))
    #.withColumn(effective_end, lit(None).cast(TimestampType()))
    .withColumn("batch_id", lit(batch_id))\
    .dropDuplicates(["customer_id"])
)
df_silver.write.format("delta").mode("overwrite").saveAsTable("silver_customers")

df_silver.createOrReplaceTempView("src")

print("Source count:", df_silver.count())
```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 14, Finished, Available, Finished, False)

Source count: 200

```
In [ ]: df_silver.show()
```

Let's see what we've staged in the temp table now

```
In [ ]: df_temp = spark.sql("SELECT * FROM src LIMIT 1000").orderBy("customer_id")
display(df_temp)
```

3. Debug - table locations

```
In [ ]: print("target table:", target_table)
print("target table path:", target_path)
```

Create target delta table if does not exist

```
In [ ]: spark.sql(f"""
CREATE TABLE IF NOT EXISTS {target_table} (
    row_sk BIGINT,
    customer_id STRING,
    customer_name STRING,
    customer_status STRING,
```

```

        address STRING,
        {effective_start} TIMESTAMP,
        {effective_end} TIMESTAMP,
        {is_current} BOOLEAN
    )
    USING DELTA
    """)

```

Capture source and change metrics to log

```

In [17]: %%sql
-- Source row count
CREATE OR REPLACE TEMP VIEW v_source_count AS
SELECT COUNT(*) AS rows_source FROM bronze_customers;

```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 19, Finished, Available, Finished, False)

Out[17]: <Spark SQL result set with 0 rows and 0 fields>

```

In [18]: %%sql
-- New vs existing keys (Lite estimation)
CREATE OR REPLACE TEMP VIEW v_change_metrics AS
SELECT
    SUM(CASE WHEN t.customer_id IS NULL THEN 1 ELSE 0 END) AS rows_inserted,
    SUM(CASE WHEN t.customer_id IS NOT NULL THEN 1 ELSE 0 END) AS rows_updated
FROM bronze_customers s
LEFT JOIN gold_dim_customer t
    ON s.customer_id = t.customer_id
    AND t.is_current = true;

```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 20, Finished, Available, Finished, False)

Out[18]: <Spark SQL result set with 0 rows and 0 fields>

```

In [ ]: '''
%%sql
ALTER TABLE gold_dim_customer ADD COLUMNS (
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    batch_id STRING
);
'''

```

Conditional alter of columns in table

```

In [ ]: table_name = "gold_dim_customer"

existing_columns = [c.name for c in spark.table(table_name).schema]

columns_to_add = {
    "created_at": "TIMESTAMP",
    "updated_at": "TIMESTAMP",
    "batch_id": "STRING"
}

for col_name, col_type in columns_to_add.items():
    if col_name not in existing_columns:
        spark.sql(f"""

```

```

        ALTER TABLE {table_name}
        ADD COLUMNS ({col_name} {col_type})
        """)
    print(f"Added column: {col_name}")
else:
    print(f"Column already exists: {col_name}")

```

```

In [ ]: print(target_table)
        spark.sql(f" select * from {target_table} ").orderBy("customer_id").show()

```

4. Define MERGE Logic (SCD2 Core Logic)

This is the **production pattern**.

The "OR" join(...): "OR" is specified as the part that joins the iterated values returned from the list comprehension (expression resolved from the list)

Concatenates them into one single string, inserting " OR " between each element

"tgt.customer_name <> src.customer_name OR tgt.address <> src.address OR tgt.status <> src.status"

Encapsulate the SCD operation within a graceful error handling encasement

```

In [ ]: scd_update_set = ",\n          ".join(
        [f"tgt.{c} = src.{c}" for c in scd_columns]
    )

    print(scd_update_set)

```

Achieving a true SCD Type-2 structure in Delta Lake

Step 1 — Close changed records

- Detect attribute changes (e.g., address, marital status) from the incoming source
- Mark previous rows as no longer current using an end date and current flag

Step 2 — Insert new versioned rows

- Insert entirely new business keys not yet in the dimension
- Insert new historical versions for changed records with a surrogate key

```

In [23]: # iterate through the tracked columns to build the dynamic 'updates'

    ...
    scd_update_set = ",\n          ".join(
        [f"tgt.{c} = src.{c}" for c in scd_columns]
    )
    ...
    ...
    {" OR ".join([f"tgt.{c} <> src.{c}" for c in scd_columns])}

    change_condition = " OR ".join(

```

```

        [f"NOT (tgt.{c} <=> src.{c})" for c in scd_columns]
    )
    ...

# -----
# Two-Step SCD Type 2 Merge (Friendly Guide Version)
# -----

# Step 0: Build null-safe change detection condition for tracked SCD columns
change_condition = " OR ".join([f"NOT (tgt.{c} <=> src.{c})" for c in scd_columns])
# ^ Checks if any tracked column changed, safely handling NULLs using `<=>`

# -----
# Step 1: Close changed records (mark as no longer current)
# -----

try:
    # Merge source into target to close rows that have changes
    spark.sql(f"""
    MERGE INTO {target_table} AS tgt
    USING (
        SELECT *, current_timestamp() AS update_ts -- timestamp for closing rec
        FROM src
    ) AS src
    ON tgt.{business_key} = src.{business_key} -- match on business key
    AND tgt.{is_current} = TRUE -- only current records
    WHEN MATCHED AND ({change_condition}) -- only if any column change
    THEN UPDATE SET
        tgt.{effective_end} = src.update_ts, -- close old record
        tgt.{is_current} = FALSE, -- mark as no longer current
        tgt.{updated_at} = current_timestamp(), -- track update timestamp
        tgt.{batch_id_flag} = '{batch_id}' -- log batch id
    """)
    status = "SUCCEEDED"
    error_message = None
except Exception as e:
    status = "FAILED"
    error_message = str(e)
    raise
finally:
    # Update ETL Log for Step 1
    spark.sql(f"""
    UPDATE etl_run_log
    SET
        end_time = current_timestamp(), -- ste
        status = '{status}', -- su
        rows_updated = (SELECT rows_updated FROM v_change_metrics), -- up
        error_message = {f"'{error_message}'" if error_message else "NULL"} -
    WHERE batch_id = '{batch_id}' -- b
    """)

# -----
# Step 2: Insert new + changed records (with surrogate key)
# -----

try:
    # Insert new rows or the versions just closed in Step 1
    spark.sql(f"""
    INSERT INTO {target_table} (
        row_sk, -- surrogate key

```

```

        {business_key},                -- business key
        {'', '.join(scd_columns)),    -- tracked SCD columns
        {effective_start},           -- record effective start
        {effective_end},             -- record effective end (NULL
        {is_current},               -- mark as current row
        {created_at},               -- creation timestamp
        {updated_at},               -- last updated timestamp
        {batch_id_flag}             -- batch identifier
    )
    SELECT
        src.row_sk,                  -- new surrogate key
        src.{business_key},          -- business key
        {'', '.join([f'src.{c}' for c in scd_columns])}, -- tracked columns from
        current_timestamp() AS {effective_start}, -- start now
        NULL AS {effective_end},     -- open-ended until changed
        TRUE AS {is_current},        -- mark as current
        current_timestamp() AS {created_at}, -- creation timestamp
        current_timestamp() AS {updated_at}, -- last update timestamp
        '{batch_id}'                 -- batch id for this insert
    FROM src
    LEFT JOIN {target_table} tgt
        ON tgt.{business_key} = src.{business_key} -- join on existing current
        AND tgt.{is_current} = TRUE                -- only current rows
    WHERE tgt.{business_key} IS NULL                -- insert only if no current
    """)
    status = "SUCCEEDED"
    error_message = None
except Exception as e:
    status = "FAILED"
    error_message = str(e)
    raise
finally:
    # Update ETL Log for Step 2
    spark.sql(f"""
        UPDATE etl_run_log
        SET
            end_time = current_timestamp(), -- ma
            status = '{status}',             -- s
            rows_inserted = (SELECT rows_inserted FROM v_change_metrics), -- i
            rows_source = (SELECT rows_source FROM v_source_count), -- t
            error_message = {f"'{error_message}'" if error_message else "NULL"} --
        WHERE batch_id = '{batch_id}'
    """)

```

StatementMeta(, 80e94774-d98e-48da-ba3d-f8a89bdc9418, 25, Finished, Available, Finished, False)

```
In [ ]: df = spark.sql("SELECT * FROM gold_dim_customer order by customer_id LIMIT 1000")
display(df)
```

```
In [ ]: %%sql
--SELECT rows_source FROM v_source_count
--SELECT rows_inserted FROM v_change_metrics
SELECT rows_updated FROM v_change_metrics
```

Fail fast validation - After the merge now, do we have some rows that may still be erroneous

```
In [ ]: dup_count = spark.sql("""
SELECT COUNT(*) AS c FROM (
```



```

SELECT customer_id
FROM gold_dim_customer
WHERE is_current = true
GROUP BY customer_id
HAVING COUNT(*) > 1
)
""").collect()[0]["c"]

if dup_count > 0:
    raise Exception("Validation failed: duplicate current records detected.")

```

End of SCD upsert run - log update

```

In [ ]: '''
%%sql
UPDATE etl_run_log
SET
    end_time = current_timestamp(),
    status = 'SUCCEEDED',
    rows_source = (SELECT rows_source FROM v_source_count),
    rows_inserted = (SELECT rows_inserted FROM v_change_metrics),
    rows_updated = (SELECT rows_updated FROM v_change_metrics)
WHERE batch_id = '{batch_id}';
'''

```

```

In [ ]: spark.sql("""
SELECT *
FROM gold_dim_customer
ORDER BY customer_id
""").show()

```

5. SCD2 Logic Explained Quickly

✓ When values change:

1. The current record is **closed**

```
* effective_end_date = current_timestamp()
```

- `is_current = false`

2. A new record is **inserted**

```
* effective_start_date = current_timestamp()
```

- `is_current = true`

✓ When values do NOT change:

- No modifications are made.

✓ When a business key does not exist:

- A new SCD2 record is inserted.

6. Check Final Results

```
In [ ]: df_dim = spark.read.format("delta").load(target_path)
display(df_dim.orderBy("customer_id", effective_start))
```

Check statuses

```
In [ ]: spark.sql("""
SELECT *
FROM gold_dim_customer
WHERE is_current = TRUE
ORDER BY customer_id, effective_start_date
""").show()
```

WHERE is_current = FALSE

```
In [ ]: spark.sql("""
SELECT *
FROM gold_dim_customer
WHERE is_current = FALSE
ORDER BY customer_id, effective_start_date
""").show()
```

```
In [ ]: # Count number of changes per customer

spark.sql("""
SELECT customer_id, COUNT(*) AS num_versions
FROM gold_dim_customer
GROUP BY customer_id
HAVING COUNT(*) > 1
ORDER BY num_versions DESC
""").show()
```

Compare consecutive rows for each customer_id to see which attributes changed.

```
In [ ]: spark.sql("""
SELECT a.customer_id, a.customer_name, b.customer_name as customer_name_stg, a.c
FROM gold_dim_customer a left outer join (select * from src) b on b.customer_id
WHERE a.is_current = FALSE
ORDER BY a.customer_id, a.effective_start_date
""").show()

# Compare consecutive rows for each customer_id to see which attributes changed.
```

```
In [ ]: spark.sql("select * from src").orderBy("customer_id").show()
```

```
In [ ]: spark.sql("""
SELECT customer_id, customer_name, customer_status, effective_start_date
FROM gold_dim_customer
WHERE effective_end_date IS NOT NULL
ORDER BY effective_end_date DESC
""").show()

# Lists only the rows that were replaced by a new version.
```

This is a fully working SCD Type 2 Delta Lake notebook.

It follows production best practices:

- Proper MERGE logic
- Versioning metadata
- Automatic start/end timestamps
- Change detection only on relevant columns
- Handles inserts + updates + new versions safely

Return Success to Pipeline

```
In [ ]: from notebookutils import mssparkutils  
mssparkutils.notebook.exit("SCD2_SUCCESS")
```